Classes
A Deeper Look
Part 1

Contents

Preprocessor wrapper

Three types of “handles” on an object

1. Name of an object
2. Reference to an object
3. Pointer to an object

Class functions

1. Predicate functions
2. Utility functions

Constructor

1. Passing arguments to constructors
2. Using default arguments in a constructor

Destructor

Friend Function

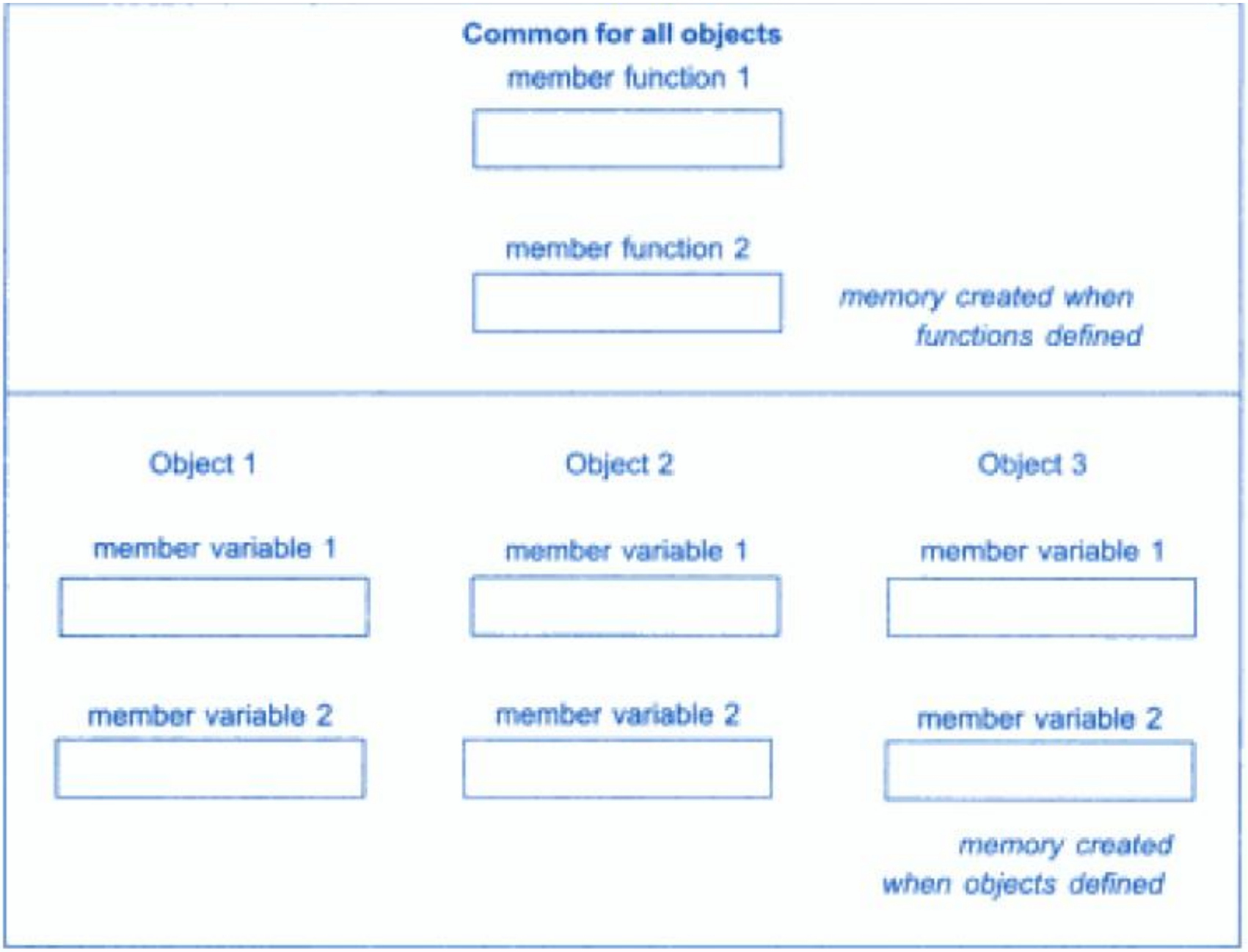
Performance Tip

1. Objects contain only data, so objects are much smaller than if they also contained member functions.

2. The compiler creates one copy (only) of the member functions separate from all objects of the class. All objects of the class share this one copy.

3. Each object, of course, needs its own copy of the class's data, because the data can vary among the objects.

4. The function code is non-modifiable (also called reentrant code or pure procedure) and, hence, can be shared among all objects of one class.


Common for all objects
member function 1
member function 2
memory created when functions defined
Object 1
Object 2
Object 3
member variable 1
member variable 1
member variable 1
member variable 2
member variable 2
member variable 2
memory created when objects defined

Class Scope and Accessing Class Members

- Dot member selection operator (.)
  - Accesses the object's members
  - Used with an object's name or with a reference to an object
- Arrow member selection operator (→)
  - Accesses the object's members
  - Used with a pointer to an object

Access Functions and Utility Functions

- Access functions
  - Can read or display data
  - Can test the truth or falsity of conditions
    - Such functions are often called predicate functions

- Utility functions (also called helper functions)
  - private member functions that support the operation of the class's public member functions
  - Not part of a class's public interface
    - Not intended to be used by clients of a class

```
#include<iostream>
using namespace std;
class item
{
private:
    int number;
    float cost;
    void read();
public:
    float Bill()
    {       return number*cost;
    }
    void setvalue(int x, float y)
    {       read();
        number = x;
        cost = y;
    }
    void Display();
};
void item::Display()
{
    cout << "Values for class members are:" << endl;
    cout << "Number =  " << number << endl;
    cout << "Cost =  " << cost << endl;
}
```

```
void item::read()
{
    cout << "Welcome" << endl;
}
```

```cpp
#include<iostream>
#include "k.h"
using namespace std;
int main()
{    item i1;// create i1 object
     item *i=&i1;//create pointer to i1
item &i2=i1;//create reference to i1
i1.setvalue(5, 9);
i1.Display();
cout << i1.Bill() << endl;
i->setvalue(6,85);
i->Display();
cout << i->Bill() << endl;
system("pause");
return 0;
}
```

Constructor

1. A constructor is a special member function whose task is to initialize the object of its class.

2. It is special because its name is same as the class name.

3. The constructor is invoked whenever an object of its associated class is created.

4. It is called constructor because it constructs the values of data members of the class.

```
#include<iostream>
using namespace std;
class integer{
private:
    int m, n;
public:
    integer();

};
integer::integer(void)
{
    m = 0, n = 0;
}
```

```
int main()
{
    integer int1;
}
```

Special Characteristics of Constructors

1. They should be declared in the public section.

2. They are invoked automatically when the objects are created.

3. They do not have return types, not even void and therefore, and they cannot return values.

Default Constructor

```
#include<iostream>

using namespace std;


class  point
{
public:
    double x, y;
    point()
    {
        x = 0; y = 0;
        cout << "Point instance created" << endl;
        //cout << x << endl << y << endl;

    }
};
```

```
int main()
{
        point p;//point instance created
    cout << p.x << endl << p.y << endl;
    cin.ignore();
    cin.clear();
    return 0;
}
```

Parameterized Constructor

1. C++ permits passing arguments to the constructor function, when the objects are created.

2. The constructors that can take arguments are called parameterized constructors.

We can modify integer constructor as follows:

```
#include<iostream>
using namespace std;
class integer{
private:
    int m, n;
public:
    integer(int x, int y);
    
};
integer::integer(int x,int y)
{
    m = x, n = y;
}
```

Parameterized Constructor

```
#include<iostream>

using namespace std;


class  point
{
public:
    double x, y;
    point(double nx, double ny)
    {
        x = nx; y = ny;
        cout << "2-parameter constructor" << endl;
        //cout << x << endl << y << endl;


    }
};
```

```
int main()
{
        point p(2.0,3.0);//point instance created
    cout << p.x << endl << p.y << endl;
    cin.ignore();
    cin.clear();
    return 0;
}
```

Uses of Parameterized Constructor

- It is used to initialize the various data elements of different objects with different values when they are created.
- It is used to overload constructors.

- **Can we have more than one constructors in a class?**

Yes, It is called **Constructor Overloading.**

Special Note

Whenever we define one or more parametric constructors for a class, a default constructor(without parameters)should also be explicitly defined as the compiler will not provide a default constructor in this case.

However, it is not necessary but it's considered to be the best practice to always define a default constructor.

Copy Constructor

A copy constructor is a member function which initializes an object using another object of the same class. A copy constructor has the following general function prototype:

```
ClassName (const ClassName &old_obj);
```

```cpp
#include<iostream>
using namespace std;
class Point
{
private:
    int x, y;
public:
    Point(int x1, int y1) { x = x1; y = y1; }

    // Copy constructor
    Point(const Point &p2) { x = p2.x; y = p2.y; }

    int getX()      { return x; }
    int getY()      { return y; }
};
```

```
int main()
{   Point p1(10, 15); // Normal constructor is called here
    Point p2 = p1; // Copy constructor is called here
    // Let us access values assigned by constructors
    cout << "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();
    cout << "\np2.x = " << p2.getX() << ", p2.y = " << p2.getY();
    system("pause");
    return 0;
}
```

Multiple Constructors in a Class

So far we have used two kind of constructors. They are:

integer(); //No arguments

integer(int,int); // two arguments

1. In the first case, the constructor itself supplies the data values and no values are passed by the calling program.
2. In the second case, the function call passes the appropriate values from main().
3. C++ permits us to use both these constructors in the same class.

```
class integer{
private:
    int m, n;
public:
    integer()
    {       m = 0; n = 0;
    }
    integer(int x, int y)
    {
        m = x; n = y;
    }
    integer(integer &i)
    {
        m = i.m; n = i.n;
    }
};
```

A class may have multiple constructors

```
#include<iostream>
using namespace std;
class  point
{
public:
    double x, y;
    point()
    {
        x = 0.0;
        y = 0.0;
        cout << "default constructor:" << endl;
    }
    point(double nx, double ny)
    {
        x = nx; y = ny;
        cout << "2-parameter constructor" << endl;
        //cout << x << endl << y << endl;

    }
};
```

```
int main()
{
        point p(2.0,3.0),p1;//point instance created
    cout << p.x << endl << p.y << endl;
    cout << p1.x << endl << p1.y << endl;
    cin.ignore();
    cin.clear();
    return 0;
}
```

Destructors

1. Destructors have the same name as that of class preceded by ~(tilde) sign.

2. Similar to constructors , destructors do not have return type not even void.

3. Only one destructor can be defined in a class , destructors do not have any parameters.

```
#include<iostream>
using namespace std;
class item
{
private:
    int number;
    float cost;
    public:
    float Bill()
    {
        return number*cost;
    }

   item(int x, float y)
   {
       number = x;
       cost = y;
   }
   void Display();
   ~item()
   {
       cout << "In Destructor" << endl;
   }
};
```

```
void item::Display()
{

    cout << "Values for class members are:" << endl;
    cout << "Number =  " << number << endl;
    cout << "Cost =  " << cost << endl;

}
int main()
{
    item i(2, 45.5);
    i.Display();
    i.~item();
system("pause");
return 0;
}
```

Friend Function

1. It is not in the scope of the class to which it is declared as friend.

2. It cannot be called using the object of the class. It can be invoked like a normal function.

3. Unlike member functions , it cannot access the member names directly and has to use an object name and dot membership operator with each member name.(A.x)

4. It can be declared either in public or private part of a class without affecting the meaning. Usually, it has objects as arguments.

5. Member function of one class can be friend function of another class.

```
#include<iostream>

using namespace std;
class sample{
    int a;
    int b;
public:
    void setvalue()
    {
        a = 45;
        b = 60;
    }
    friend float mean(sample s);


};
```

```
float mean(sample s)
{
    return float(s.a + s.b) / 2;
}
int main()
{

    sample x;
    x.setvalue();
    cout << "Mean Value=" << mean(x) << endl;
    cin.ignore();
    cin.clear();
    return 0;
}
```

How to declare a function as friend of a class

1. To declare a function as a friend of a class

   – Provide the function prototype in the class definition preceded by keyword friend

2. To declare a class as a friend of a class.

   we can also declare all members of one class as the friend function of another class .In such cases, the class is called a friend class.

```cpp
#include<iostream>

using namespace std;
class ABC;
class XYZ
{
private:
    int x;
public:
    void setvalue(int i)
    {
        x = i;
    }
    friend void max(XYZ, ABC);

};
```

```cpp
class ABC{
 private:
     int a;
 public:
     void setvalue(int i)
     {
         a = i;

     }

     friend void max(XYZ, ABC);
 };
```

```
void max(XYZ m, ABC n)
{
        if (m.x >= n.a)
                cout << m.x;
        else
                cout << n.a;
}

int main()
{
        ABC abc;
        abc.setvalue(10);
        XYZ xyz;
        xyz.setvalue(20);
        max(xyz, abc);
        cin.ignore();
        cin.clear();
        return 0;
}
```

Some important point about friend Functions and friend Classes

- Friendship is granted, not taken

  For class B to be a friend of class A, class A must explicitly declare that class B is its friend

- Friendship relation is neither symmetric nor transitive

  If class A is a friend of class B, and class B is a friend of class C, you cannot infer that class B is a friend of class A, that class C is a friend of class B, or that class A is a friend of class C

Friend Class Example

```
#include<iostream>

using namespace std;
class mysecondclass;
class firstclass{
    friend class mysecondclass;
private:
    int secret;
public:
    firstclass()
    {
        secret = 0;
            }
    void printmember()
    {
        cout << secret << endl;
    }
};
```

```
class mysecondclass
{
public:
    void change(firstclass f, int x)
    {
        f.secret = x;
        cout << f.secret << endl;;
    }
};

int main()
{

    firstclass f1;
    mysecondclass s1;
    f1.printmember();
    s1.change(f1, 5);
        cin.ignore();
    cin.clear();
    return 0;
}
```

Observations

Even though the prototypes for friend functions appear in the class definition, friends are not member functions.

Adding Constructors to a class UML